Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών
Μάθημα: ″Σχεδιασμός Ενσωματωμένων Συστημάτων″
Ακαδ. Έτος: 2026-2027

1η Εργαστηριακή Άσκηση

Βελτιστοποίηση Αλγορίθμων για Χαμηλή Κατανάλωση Ενέργειας και Υψηλή Απόδοση

Αποστολόπουλος Νικόλαος
(03122614)
Γκριμπογιάννης Χαρίλαος
(03122002)

Περίληψη

Η παρούσα εργασία υλοποιήθηκε στα πλαίσια του μαθήματος ″Σχεδιασμός Ενσωματωμένων Συστημάτων″ και αφορά ...

Περιεχόμενα

1 Βελτιστοποίηση της Εφαρμογής PHODS **3**
1.1 Χαρακτηριστικά του Board 3
1.2 Μέτρηση Χρόνου Εκτέλεσης 4
1.3 Μετασχηματισμοί Κώδικα 4
1.4 Εξερεύνηση Μεγέθους Μπλοκ $B \times B$ 6
1.5 Εξερεύνηση Μεγέθους Μπλοκ $B_x \times B_y$ 6

2 Βελτιστοποίηση της Εφαρμογής tables.c **6**
2.1 Σύγκριση Αρχιτεκτονικών 7
2.2 Loop Unrolling 7
2.3 Μέγεθος L1 Data Cache 8
2.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση 9
2.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου 11

1 Βελτιστοποίηση της Εφαρμογής PHODS

1.1 Χαρακτηριστικά του Board

Καταγράφουμε τα βασικά χαρακτηριστικά του μικροελεγκτή και του board, συνδυάζοντας το datasheet της οικογένειας Renesas S7G2 και το περιβάλλον e2 studio.

Μνήμες.

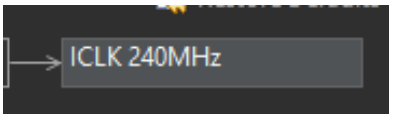
- **RAM (SRAM):** 640 kB συνολικά. Τα πρώτα 32 kB διαθέτουν προστασία Double-bit Error Detection (DED), ενώ η υπόλοιπη περιοχή (608 kB) ελέγχεται με Parity check (Table 1.2 και Table 1.16 του datasheet).
- **Code Flash:** 4 MB, για το part number του board R7FS7G27H3A01CFC (Table 1.15 του datasheet).
- **Data Flash:** 64 kB.

Κρυφές Μνήμες. Το board διαθέτει Flash Cache (FCACHE), η οποία λειτουργεί ως ενδιάμεση μνήμη υψηλής ταχύτητας ανάμεσα στον πυρήνα ARM Cortex-M4 και την κύρια μνήμη Code Flash. Αποθηκεύει προσωρινά εντολές και δεδομένα που προσπελαύνονται επαναλαμβανόμενα, μειώνοντας τους κύκλους αναμονής (wait states) και επιταχύνοντας την εκτέλεση. Αποτελείται από τρεις μονάδες (Πίνακας 1), οι οποίες καλύπτουν τις διευθύνσεις `0000 0000h–003F FFFFh`.

	FCACHE1	**FCACHE2**	**FLPF**
Προορισμός	CPU instruction fetch	CPU operand access, DMA	Prefetch (FLPF)
Χωρητικότητα	256 bytes	16 bytes	32 bytes
Associativity	8-way set associative	Fully associative	Fully associative
Μέγεθος entry	128 bits	128 bits	128 bits
Entries	2 ανά way	1	2
Cache hit	0 waits	0 waits	0 waits
Cache miss	Waits όπως ορίζονται στο Flash Wait Cycle Register		

Πίνακας 1: Flash Cache του S7G2

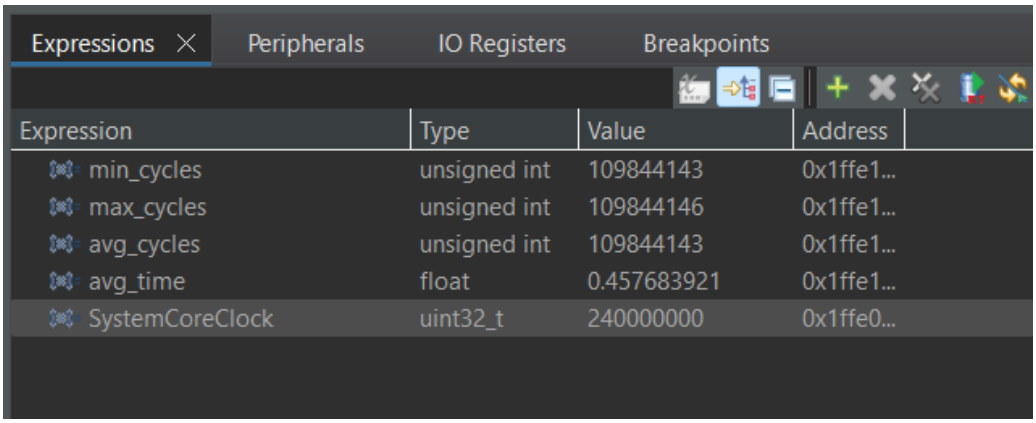
Συχνότητα Ρολογιού. Η συχνότητα του κεντρικού ρολογιού (`SystemCoreClock` / ICLK) είναι 240 MHz. Η τιμή επιβεβαιώθηκε τόσο από τα χαρακτηριστικά μέγιστης λειτουργίας του πυρήνα στο datasheet, όσο και από το e2 studio, όπως φαίνεται στο Σχήμα 1 (καρτέλα Clocks Configuration).Η ίδια τιμή φαίνεται και στο Σχήμα 2, όπου το watch expression `SystemCoreClock` δείχνει 240000000.



Σχήμα 1: Συχνότητα ICLK στο Clocks Configuration του e2 studio

1.2 Μέτρηση Χρόνου Εκτέλεσης

Ο χρόνος εκτέλεσης της συνάρτησης `phods_motion_estimation` του αρχικού (μη βελτιστοποιημένου) κώδικα μετρήθηκε διαβάζοντας τον καταχωρητή `DWT->CYCCNT` πριν και μετά την κλήση. Η διαφορά των δύο τιμών δίνει τους κύκλους ρολογιού και διαιρώντας με τη συχνότητα των 240 MHz προκύπτει ο χρόνος. Η διαδικασία επαναλήφθηκε 10 φορές και στο Σχήμα 2 φαίνονται οι τιμές που κατέγραψε ο debugger.



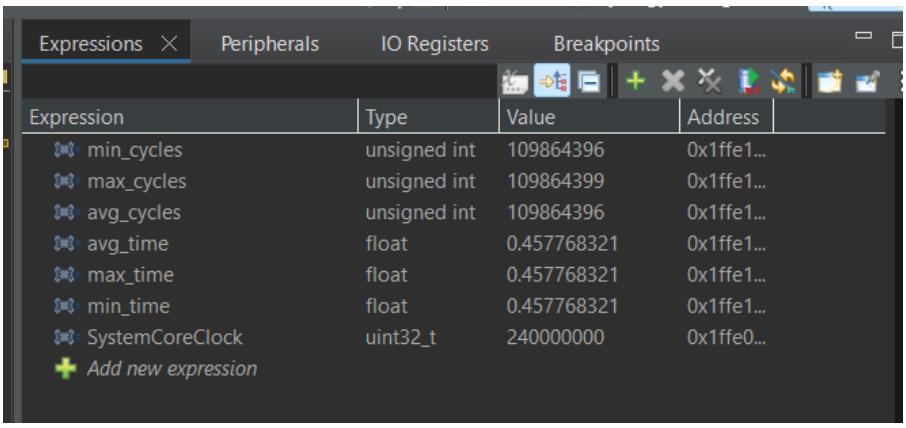
Σχήμα 2: Αποτελέσματα μέτρησης του αρχικού κώδικα (Expressions του e2 studio)

Ο ελάχιστος αριθμός κύκλων είναι 109 844 143 και ο μέγιστος 109 844 146, δηλαδή ο αρχικός κώδικας εκτελείται σε περίπου 0,4577 s. Παρατηρήσαμε ότι μόνο η πρώτη εκτέλεση απαιτεί 109 844 146 κύκλους, ενώ και οι υπόλοιπες εννέα απαιτούν 109 844 143. Η ύπαρξη διαφοράς μόνο στην πρώτη εκτέλεση οφείλεται πιθανότατα στην Flash Cache, η οποία είναι ακόμη κενή και προκαλεί επιπλέον cache misses, ενώ από τη δεύτερη εκτέλεση και μετά περιέχει ήδη τα δεδομένα.

1.3 Μετασχηματισμοί Κώδικα

Εφαρμόζουμε διαδοχικά τρεις μετασχηματισμούς στον αρχικό κώδικα και μετράμε κάθε φορά τον χρόνο εκτέλεσης με την ίδια διαδικασία (10 εκτελέσεις). Κάθε μετασχηματισμός εφαρμόζεται πάνω στον κώδικα της προηγούμενης έκδοσης.

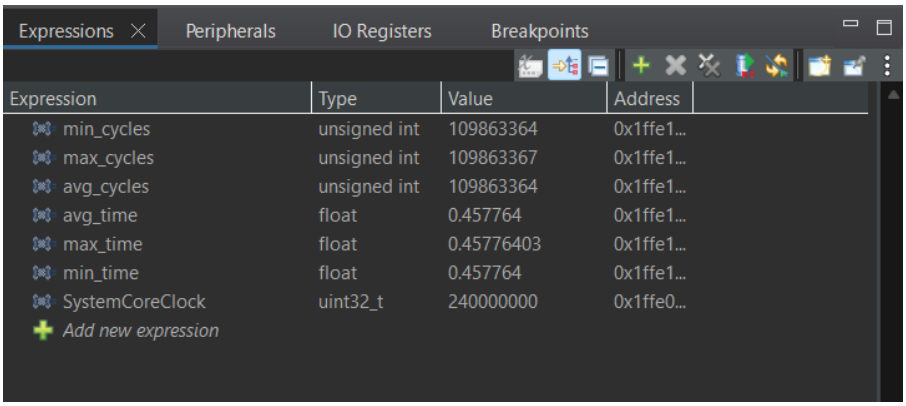
Loop Unrolling. Ο εξωτερικός βρόχος `while(S > 0)` εκτελείται ακριβώς τρεις φορές, με S = 4, 2 και 1. Τον ξετυλίγουμε πλήρως σε τρία διαδοχικά αντίγραφα του σώματός του, με σκοπό την αποφυγή του κόστους ελέγχου και ενημέρωσης του βρόχου. Στο Σχήμα 3 φαίνονται τα αποτελέσματα.



Σχήμα 3: Αποτελέσματα μέτρησης μετά το Loop Unrolling

Ο ελάχιστος αριθμός κύκλων είναι 109 864 396 και ο μέγιστος 109 864 399, δηλαδή ο χρόνος εκτέλεσης είναι 457,768 ms έναντι 457,684 ms του αρχικού κώδικα. Ο μετασχηματισμός δεν βελτιώνει την απόδοση, αντίθετα την επιβαρύνει ελάχιστα (+20 253 κύκλοι, περίπου 0,02%). Ο βρόχος που ξετυλίξαμε εκτελείται μόλις τρεις φορές ανά block, οπότε το κόστος του είναι αμελητέο σε σχέση με το κόστος των εσωτερικών βρόχων. Επιπλέον, ο κώδικας τριπλασιάζεται σε μέγεθος, κάτι που πιθανώς επιβαρύνει τις προσπελάσεις εντολών από τη Flash.

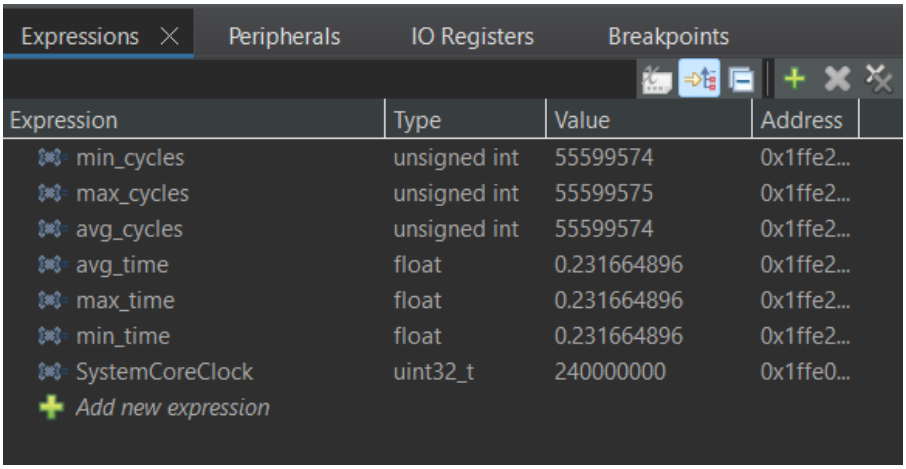
Loop Reversal. Στους εσωτερικούς βρόχους των δεικτών k και l αντιστρέφουμε τη φορά διαδρομής, ώστε οι δείκτες να μειώνονται μέχρι το 0 αντί να αυξάνονται μέχρι το B, με στόχο τη σύγκριση με το μηδέν. Δεν εφαρμόσαμε Loop Interchange, καθώς οι πίνακες της C αποθηκεύονται κατά γραμμές και οι υπάρχοντες βρόχοι ήδη διατρέχουν τα δεδομένα γραμμή προς γραμμή, οπότε η εναλλαγή θα κατέστρεφε τη χωρική τοπικότητα. Τα αποτελέσματα φαίνονται στο Σχήμα 4.



Σχήμα 4: Αποτελέσματα μέτρησης μετά το Loop Unrolling και το Loop Reversal

Ο ελάχιστος αριθμός κύκλων είναι 109 863 364 και ο μέγιστος 109 863 367, δηλαδή 457,764 ms. Σε σχέση με την προηγούμενη έκδοση το κέρδος είναι μόλις 1 032 κύκλοι (περίπου 0,001%), ενώ σε σχέση με τον αρχικό κώδικα ο χρόνος παραμένει λίγο χειρότερος. Η βελτίωση είναι ουσιαστικά αμελητέα, αφού το κόστος εκτέλεσης δεν προέρχεται από τον έλεγχο του δείκτη των βρόχων αλλά από το σώμα τους, δηλαδή από τις προσπελάσεις των πινάκων και τους ελέγχους ορίων.

Data Reuse Transformation (DRT). Οι πίνακες `current` και `previous` είναι `const`, άρα βρίσκονται στη μνήμη Flash, της οποίας οι προσπελάσεις απαιτούν κύκλους αναμονής σε περίπτωση cache miss. Επειδή η Flash Cache του board είναι πολύ μικρή, σχεδόν κάθε ανάγνωση pixel πληρώνει αυτό το κόστος. Για κάθε block αντιγράφουμε μία φορά σε πίνακες της RAM το block του `current` (B×B) και το παράθυρο αναζήτησης του `previous` ((B+2p)×(B+2p)). Οι υπολογισμοί γίνονται έπειτα αποκλειστικά πάνω στα αντίγραφα της RAM. Κατά την αντιγραφή του παραθύρου τα pixels εκτός εικόνας τίθενται στο 0, οπότε οι έλεγχοι ορίων δεν χρειάζονται πλέον μέσα στον εσωτερικό βρόχο. Στο Σχήμα 5 φαίνονται τα αποτελέσματα.



Σχήμα 5: Αποτελέσματα μέτρησης μετά το Loop Unrolling, το Loop Reversal και το DRT

Ο ελάχιστος αριθμός κύκλων είναι 55 599 574 και ο μέγιστος 55 599 575, δηλαδή ο χρόνος εκτέλεσης μειώνεται σε 231,665 ms. Σε σχέση με τον αρχικό κώδικα (457,684 ms) η μείωση είναι περίπου 49%, δηλαδή speedup 1,98×. Η μεγάλη βελτίωση οφείλεται κυρίως στη μεταφορά των δεδομένων από τη Flash στη RAM, καθώς κάθε pixel διαβάζεται πλέον από τη Flash μία φορά ανά block και όχι σε κάθε επανάληψη των υπολογισμών, ενώ συνεισφέρει και η αφαίρεση των ελέγχων ορίων από τον εσωτερικό βρόχο. Αυτό εξηγεί και γιατί οι προηγούμενοι μετασχηματισμοί δεν βελτίωναν την απόδοση: το σημείο συμφόρησης ήταν οι προσπελάσεις στη Flash και όχι η επιβάρυνση των βρόχων.

1.4 Εξερεύνηση Μεγέθους Μπλοκ $B \times B$

1.5 Εξερεύνηση Μεγέθους Μπλοκ $B_x \times B_y$

2 Βελτιστοποίηση της Εφαρμογής tables.c

Στην παρούσα παράγραφο βελτιστοποιούμε την εφαρμογή tables.c (βλ. Listing 1) εκτελούμενη σε δύο διαφορετικές αρχιτεκτονικές x86 με χρήση του προσομοιωτή Gem5. Η πρώτη αρχιτεκτονική που μελετάμε αφορά μια αρχιτεκτονική επεξεργαστή και κύριας μνήμης, ενώ η δεύτερη είναι επαυξημένη με L1 Instruction Cache, L1 Data Cache και L2 Cache. Συγκεκριμένα, μελετάμε την επίδραση της τεχνικής του loop unrolling, καθώς και την επίδραση της ύπαρξης κρυφών μνημών διαφορετικών μεγεθών στον χρόνο εκτέλεσης της εφαρμογής μετρημένο σε κύκλους ρολογιού.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
```

```c
int main()
{
    int i;
    int N = 100000;
    double *x1, *x2, *x3, *y;
    double a1 = 0.5;
    double a2 = 1;
    double a3 = 1.5;

    x1 = (double*) malloc(N * sizeof(double));
    x2 = (double*) malloc (N * sizeof(double));
    x3 = (double*) malloc (N * sizeof(double));
    y = (double*) malloc (N * sizeof(double));

    //Do not modify this loop
    for (i=0; i<=N-1; i++)
    {
        x1[i] = (double) i * 0.5;
        x2[i] = (double) i * 0.8;
        x3[i] = (double) i * 0.2;
        y[i] = 0;
    }

    for (i=0; i<=N-1; i++)
    {
        y[i] = y[i] + a1*x1[i] + a2*x2[i] + a3*x3[i];
    }

    return 0;

}
```

Listing 1: Source Code (tables.c)

2.1 Σύγκριση Αρχιτεκτονικών

Αρχικά προσομοιώνουμε την εκτέλεση της εφαρμογής tables.exe σε αρχιτεκτονική που περιλαμβάνει τον επεξεργαστή και την κύρια μνήμη. Η εκτέλεση γίνεται σε 858,490,254 κύκλους. Ύστερα προσομοιώνουμε την εκτέλεση της ίδιας εφαρμογής σε σύστημα που διαθέτει επιπλέον L1 Instruction και L1 Data Caches μεγέθους 8kB και L2 Cache μεγέθους 128kB. Η εφαρμογεί εκτελείται σε 59,829,295 κύκλους. Παρατηρούμε ότι η ενσωμάτωση κρυφών μνημών στο σύστημα μείωσε τον χρόνο εκτέλεσης κατά 798,660,959 κύκλους δίνοντας ένα speedup περίπου 14.35x. Η δραματική αυτή αύξηση της ταχύτητας εκτέλεσης αποδεικνύει έμπρακτα την ανάγκη ύπαρξης των κρυφών μνημών, καθώς χωρίς αυτές κάθε ανάγνωση/εγγραφή που εκτελεί το πρόγραμμα απαιτεί πολύ αργές προσπελάσεις στην κύρια μνήμη.

2.2 Loop Unrolling

Αφού αποδείξαμε την ανωτερότητα της αρχιτεκτονικής με κρυφές μνήμες, θα μελετήσουμε πλέον μόνο αυτή την αρχιτεκτονική. Για να ελαττώσουμε περαιτέρω τον χρόνο εκτέλεσης της εφαρμογής, εισάγουμε τη μέθοδο loop unrolling και εξετάζουμε την επίδραση της εφαρμογής της.

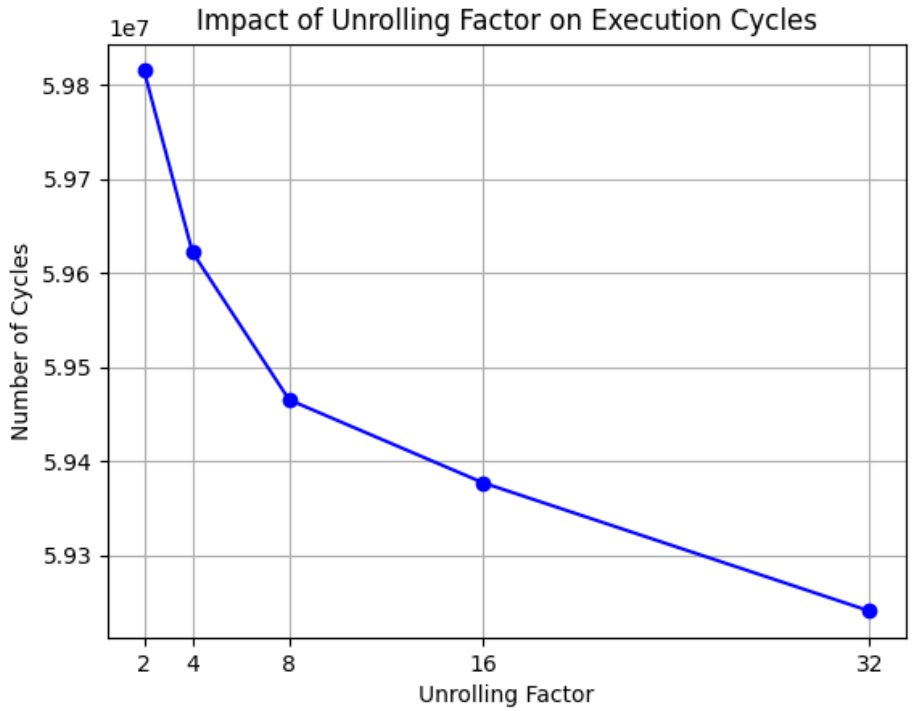
Η μέθοδος loop unrolling είναι ένας μετασχηματισμός που συνίσταται στο "ξετύλιγμα" του βρόχου. Στο παρόν την εφαρμόζουμε στον βρόχο υπολογισμού του πίνακα

y όπως φαίνεται στο Listing 2.

```
for (i = 0; i <= N - k; i = i + k) {
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
    ...
    y[(i + k)] = y[(i + k)] + a1 * x1[(i + k)] + a2 * x2[(i + k)] + a3 * x3[(i + k)];
}
for (int i = N - ((N - (0)) % k); i <= N - 1; i = i + 1)
    y[i] = y[i] + a1 * x1[i] + a2 * x2[i] + a3 * x3[i];
```

Listing 2: Loop Unrolling with Unrolling Factor k

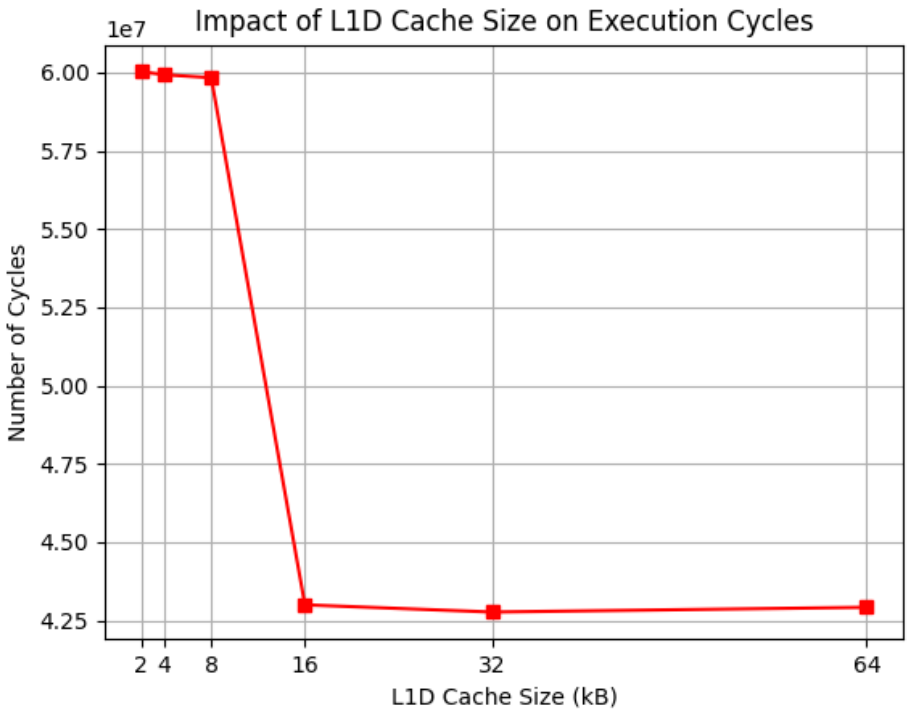
Στο Σχήμα 6 φαίνεται η επίδραση του μετασχηματισμού στον χρόνο εκτέλεσης σε πλήθος κύκλων ρολογιού για διαφορετικά unrolling factors (2, 4, 8, 16 και 32). Παρατηρούμε ότι ... Αυτό οφείλεται...



Σχήμα 6: Επίδραση του Unrolling Factor στον Χρόνο Εκτέλεσης

2.3 Μέγεθος L1 Data Cache

Συνεχίζοντας τη μελέτη της δεύτερης αρχιτεκτονικής, προχωράμε σε μια εξερεύνηση της επίδρασης διαφορετικών μεγεθών L1 Data Cache (2kB, 4 kB, 8kB, 16kB, 32kB και 64kB).



Σχήμα 7: Επίδραση του Μεγέθους της L1 Data Cache στον Χρόνο Εκτέλεσης

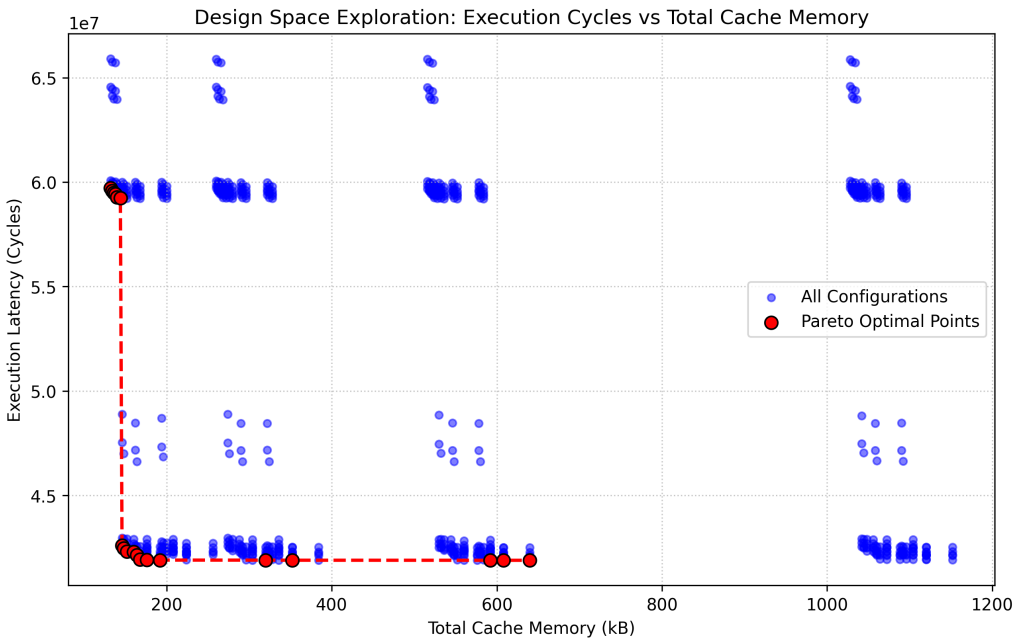
Εδώ παρατηρούμε κάτι πολύ ενδιαφέρον...

2.4 Βελτιστοποίηση με Εξαντλητική Αναζήτηση

Έχοντας μελετήσει την επίδραση συγκεκριμένων παραμέτρων της εφαρμογής (Unrolling Factor) αλλά και της αρχιτεκτονικής (Μέγεθος L1 Data Cache), έχουμε σχηματίσει μια εικόνα για τη βελτιστοποίηση. Αυτό που μας ενδιαφέρει είναι να ελαχιστοποιήσουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής αλλά και τη συνολική μνήμη που θα χρησιμοποιήσουμε (το άθροισμα των L1I, L1D και L2 σε kB). Έτσι, καταγράφουμε έναν χώρο προς εξερεύνηση, ο οποίος ορίζεται από τους συνδυασμούς των παρακάτω

L1D Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L1I Cache Size $\in$ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
L2 Cache Size $\in$ [128kB, 256kB, 512kB, 1024kB]
Unrolling Factor $\in$ [2, 4, 8, 16, 32]

Προχωράμε σε εξαντλητική αναζήτηση του χώρου εξερεύνησης και για κάθε ένα από τα 720 σημεία του χώρου καταγράφουμε το πλήθος των κύκλων που απαιτούνται για την εκτέλεση της εφαρμογής. Στο Σχήμα 8 φαίνονται οι θέσεις όλων των σημείων και το Pareto Frontier, ενώ στον Πίνακα 2 παραθέτουμε το Pareto Frontier μαζί με τα στοιχεία της αρχιτεκτονικής που οδήγησαν στο εκάστοτε Pareto Optimal σημείο.



Σχήμα 8: Exhaustive Search Pareto Frontier

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
2	2	128	8	132	59724828
2	16	128	8	146	42591707
4	2	128	16	134	59579732
4	4	128	16	136	59484646
4	16	128	16	148	42472280
4	32	128	16	164	42146735
8	2	128	32	138	59427801
8	4	128	32	140	59281539
8	8	128	32	144	59241019
8	16	128	32	152	42315832
8	32	128	32	168	41928215
16	16	128	32	160	42300132
16	32	128	32	176	41912388
16	64	512	32	592	41889209
32	32	128	32	192	41898540
32	32	256	32	320	41896021
32	64	256	32	352	41892206
32	64	512	32	608	41889106
64	64	512	32	640	41888238

Πίνακας 2: Exhaustive Search Pareto Optimal Points

Παρατηρούμε ότι... Τέλος, σημειώνουμε ότι λόγω του μεγάλου πλήθους προσομοιώσεων (720 σημεία), για την εξαντλητική αναζήτηση απαιτήθηκαν 2 ώρες και 46 λεπτά.

2.5 Βελτιστοποίηση με τη χρήση Γενετικού Αλγορίθμου

Αντί για τη χρονοβόρα εξαντλητική αναζήτηση του χώρου εξερεύνησης, μπορούμε να χρησιμοποιήσουμε έναν γενετικό αλγόριθμο για να λάβουμε το Pareto Frontier. Ο αλγόριθμος αυτός μπλα μπλα μπλα...

Εκτελώντας τον αλγόριθμο παρατηρούμε ότι κάνει μόνο 30 evaluations και τερματίζει σε 7 λεπτά και 30 δευτερόλεπτα, επιστρέφοντας το Pareto Frontier του Πίνακα 3. Παρατηρούμε αρχικά ότι ο αλγόριθμος βρήκε 3 Pareto Optimal σημεία, σε αντίθεση με τα 19 της εξαντλητικής αναζήτησης. Τέλος, παρατηρούμε ότι μόνο ένα εκ των τριών σημείων (32kB, 32kB, 256kB, 32) είναι στην πραγματικότητα Pareto Optimal και βρίσκεται στα αποτελέσματα της προηγούμενης υποπαραγράφου. Αυτό συμβαίνει καθώς ο αλγόριθμος βασίζεται σε έναν ευρετικό (heuristic) μηχανισμό...

L1I (kB)	L1D (kB)	L2 (kB)	UF	Total Memory (kB)	Latency (CC)
32kB	32kB	512kB	32	576	41895278
16kB	32kB	128kB	16	176	42130415
32kB	32kB	256kB	32	320	41896021

Πίνακας 3: Genetic Algorithm Pareto Optimal Points